Decorating Parking Charges

ICT 4315: Object Oriented Methods and Programming II

for

Master of Science

Information Technology

Kalika Browder

University of Denver College of Professional Studies

May 10, 2026

Faculty: Nathan Braun, MBA
Director: Cathie Wilson, M.S.
Dean: Michael J. McGuire, MLS

The University Parking System has evolved consistently and constantly throughout several design iterations. After implementing parking charge calculation with the Strategy pattern, this assignment challenged me to reconsider that design and then apply the Decorator pattern instead. The below describes the new design, the implementation decisions made, challenges encountered, and a critical comparison of the two approaches. I was thoroughly challenged by this assignment, but feel as if the most was made out of it, and that the new design is successful in its implementation.

Design

The Decorator pattern structures pricing as a chain of objects where each link adds exactly one pricing factor. The design centers on an abstract base class, ParkingChargeCalculator, which defines the single method getParkingCharge(Instant entryTime, ParkingLot lot, ParkingPermit permit). This method is the only interface any participant in the chain needs to honour.

FlatRateCalculator is the concrete base component, as it sits at the innermost position of every chain and returns the lot's configured base rate without modification. For ENTRY_ONLY lots this is feeOnEntry; for ENTRY_EXIT lots it is feeOvernight, which serves as the duration-based rate.

ParkingChargeCalculatorDecorator is the abstract decorator. It extends ParkingChargeCalculator (IS-A relationship) and holds a reference to another ParkingChargeCalculator (HAS-A relationship). This dual role is the structural heart of the Decorator pattern: decorators are interchangeable with the base component, yet they can wrap any other calculator transparently.

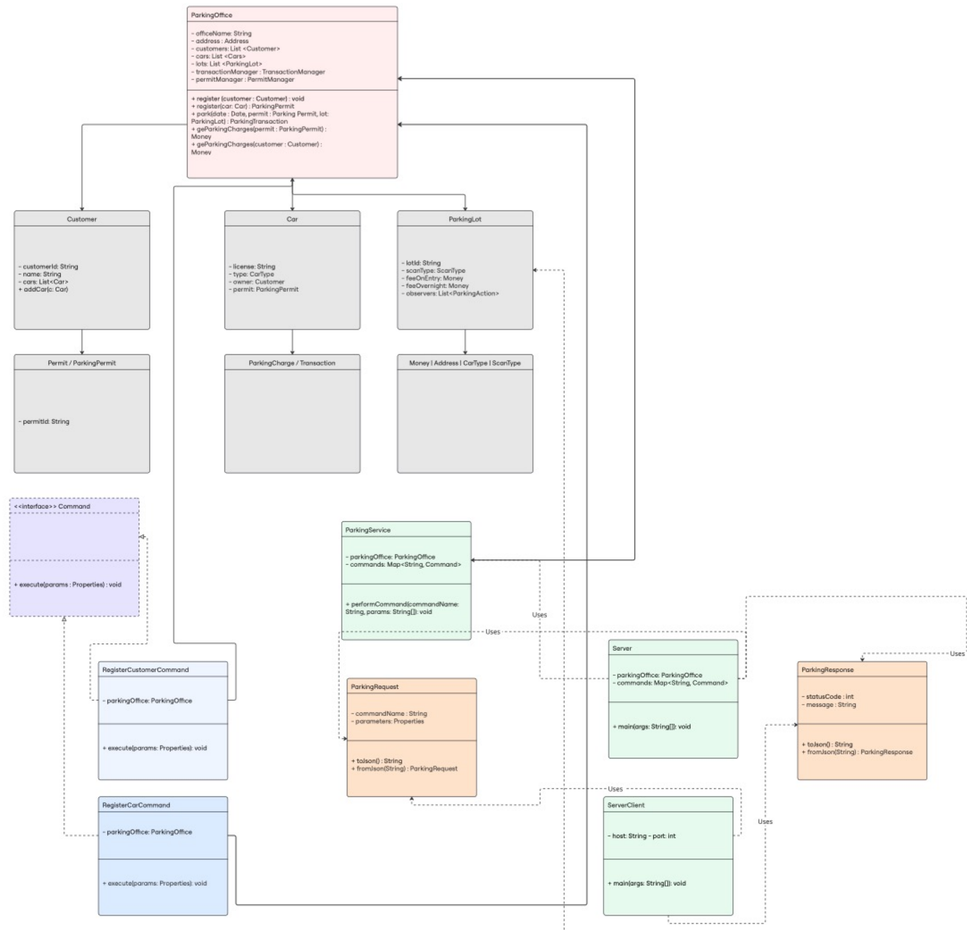
The following four concrete decorators were implemented:

- **CompactCarDiscountDecorator**: reduces the charge by 20% if the permit's vehicle is CarType.COMPACT.
- **WeekendDiscountDecorator**:reduces the charge by 15% on Saturday and Sunday.
- **PrimeTimeSurchargeDecorator**: adds a 50% surcharge during commuter prime-time windows (7–9 am and 4–6 pm).
- **SpecialDaySurchargeDecorator**:adds a 75% surcharge on registered special dates (e.g., graduation day).

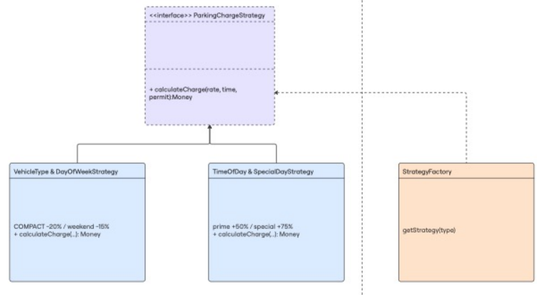
Finally, ParkingChargeCalculatorFactory is a static factory that constructs the correct chain for each ScanType. For ENTRY_ONLYlots it wraps FlatRateCalculator with CompactCarDiscountDecorator, then WeekendDiscountDecorator, then SpecialDaySurchargeDecorator. For ENTRY_EXIT lots it substitutes PrimeTimeSurchargeDecorator for the weekend decorator, reflecting the different pricing logic appropriate to duration-based parking.

UML Diagram

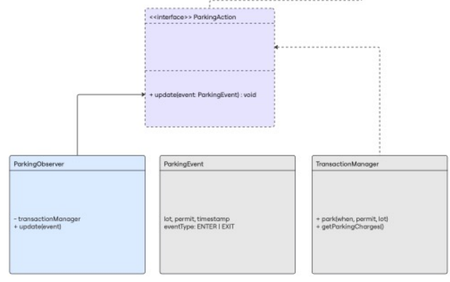
The below UML class diagram illustrates the full structure of the Decorator pattern implementation across three layers. At the top, the abstract ParkingChargeCalculator component defines the shared interface that every participant in the chain must implement. Below it, FlatRateCalculator extends it as the concrete leaf, while ParkingChargeCalculatorDecorator extends it as the abstract decorator — holding a component reference back to a ParkingChargeCalculator, which is what enables the wrapping chain. The four concrete decorators (CompactCarDiscountDecorator, WeekendDiscountDecorator, PrimeTimeSurchargeDecorator, and SpecialDaySurchargeDecorator) each extend the abstract decorator with their individual pricing factor. Finally, ParkingChargeCalculatorFactory sits at the bottom of the diagram with a dependency arrow pointing upward into the chain, reflecting that it constructs but does not inherit from any calculator class. The IS-A and HAS-A relationships visible in the diagram are what distinguish the Decorator pattern from simple inheritance. The decorator is both a ParkingChargeCalculator and contains one, allowing unlimited compositional depth.



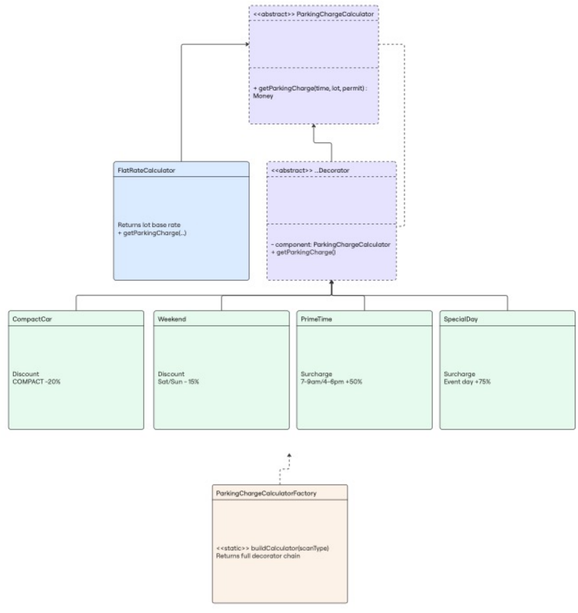
Strategy Pattern: Week 4



Observer Pattern: Week 5



Decorator Pattern: Week 6



Implementation Challenges

The first challenge was determining base rate selection. The FlatRateCalculator needed to choose between feeOnEntry and feeOvernight based on ScanType. This was resolved cleanly, however, inside FlatRateCalculator by inspecting the lot's scan type, keeping that decision in one place and out of the decorators.

The second biggest challenge for me was deciding how surcharges and discounts interact when multiple decorators are stacked. Rather than implementing a "highest factor wins" rule at the decorator level (which would require decorators to know about each other), a multiplicative application was chosen. Each decorator multiplies the value it receives from its inner component. This means a COMPACT car during prime-time on graduation day accumulates all three factors… i.e. a deliberate, transparent design choice. If "highest wins" semantics are ever needed, a single outermost decorator can implement that logic without modifying any existing class.

The third challenge was null safety. The abstract decorator throws IllegalArgumentException if constructed with a null component, failing fast at construction time rather than at runtime. Individual decorators guard against null entryTime and null permit by returning the unmodified inner charge rather than throwing exceptions.

Strategy vs. Decorator

In this instance, both patterns to me achieve open/closed design, so new pricing rules can be added without modifying existing code. However, they differ in important ways that I think I need to note.

The Strategy pattern encapsulates an entire pricing algorithm in one class. This works well when pricing rules are stable and monolithic. The weakness is that combining multiple independent rules: compact discount AND weekend discount AND prime-time surcharge, as this requires either a single large strategy class that handles all combinations, or a proliferation of strategy subclasses for every combination. In the existing codebase, VehicleTypeAndDayOfWeekStrategy already bundled two rules together, and TimeOfDayAndSpecialDayStrategy bundled two others. Adding a third rule to either would require modifying the class.

The Decorator pattern solves this problem directly. Each factor is encapsulated in its own class, and combinations are assembled by the factory at construction time. Adding a new pricing rule, say, an OvernightSurchargeDecorator, means writing one new class and updating the factory. No existing class needs modification.

The trade-off, to me, comes down to complexity. The Decorator pattern requires more classes and a more careful mental model: developers must understand the chain structure to reason about pricing outcomes. Debugging requires tracing through multiple layers. The Strategy pattern is easier to read in isolation; a single method tells the complete story of one algorithm.

For a parking system with multiple independent, composable pricing factors, the Decorator pattern is the superior choice, in my opinion. It adheres more strictly to the Single Responsibility Principle, that each class does exactly one thing, and then scales far more gracefully as new factors are introduced. The Strategy pattern remains appropriate when pricing algorithms are truly monolithic and unlikely to share factors with other algorithms. In this system, where compact discounts, time-of-day surcharges, and special-day rates are all independent dimensions that can apply simultaneously, composition through decoration is the right tool. The factory ensures that the rest of the system remains completely decoupled from the chain construction details, preserving the benefits of both patterns working in concert.